

# Algorithms and Data Structures

## Problem Sheet 9

Hermann Schichl

81. Show, by means of a counterexample, that the following “greedy” strategy does not always determine an optimal way to cut rods. Define the density of a rod of length  $i$  to be  $p_i/i$ , that is, its value per inch. The greedy strategy for a rod of length  $n$  cuts off a first piece of length  $i$ , where  $1 \leq i \leq n$ , having maximum density. It then continues by applying the greedy strategy to the remaining piece of length  $n - i$ .
82. Consider a modification of the rod-cutting problem in which, in addition to a price  $p_i$  for each rod, each cut incurs a fixed cost of  $c$ . The revenue associated with a solution is now the sum of the prices of the pieces minus the costs of making the cuts. Give a dynamic-programming algorithm to solve this modified problem.
83. Let  $R(i, j)$  be the number of times that table entry  $m[i, j]$  is referenced while computing other table entries in a call of **MATRIX-CHAIN-ORDER**. Show that the total number of references for the entire table is

$$\sum_{i=1}^n \sum_{j=i}^n R(i, j) = \frac{n^3 - n}{3}.$$

84. As stated, in dynamic programming, you first solve the subproblems and then choose which of them to use in an optimal solution to the problem. Professor Capulet claims that she does not always need to solve all the subproblems in order to find an optimal solution. She suggests that she can find an optimal solution to the matrix-chain multiplication problem by always choosing the matrix  $A_k$  at which to split the subproduct  $A_i A_{i+1} \cdots A_j$  (by selecting  $k$  to minimize the quantity  $p_{i-1} p_k p_j$ ) before solving the subproblems. Find an instance of the matrix-chain multiplication problem for which this greedy approach yields a suboptimal solution.
85. Suppose that the rod-cutting problem of Section 14.1 also had a limit  $l_i$  on the number of pieces of length  $i$  allowed to be produced, for  $i = 1, 2, \dots, n$ . Show that the optimal-substructure property described in Section 14.1 no longer holds.
86. Longest simple path in a directed acyclic graph: A directed graph  $\Gamma$  is a graph, where every edge has a direction. Mathematically, that means that  $\Gamma = (V, E)$ , with a finite set  $V$  of vertices and a finite set  $E \subseteq V \times V$  of directed edges. Note that edges of directed graphs are ordered pairs, so  $(i, j) \neq (j, i)$  if  $i \neq j$ . A path of length  $k$  in a directed graph is a sequence of vertices  $v_0 v_1 v_2 \dots v_k$ , where  $(v_{i-1}, v_i) \in E$  for all

$i = 1, \dots, k$ . A path is called a cycle, if  $v_0 = v_k$ . A directed graph without cycles is called a directed acyclic graph, short DAG.

You are given a directed acyclic graph  $\Gamma = (V, E)$  with real-valued edge weights, i.e., every edge  $e \in E$  has a weight  $w_e \in \mathbb{R}$ , and two distinguished vertices  $s$  and  $t$ . The weight of a path is the sum of the weights of the edges in the path. Describe a dynamic-programming approach for finding a longest weighted simple path from  $s$  to  $t$ . What is the running time of your algorithm?

87. Longest palindrome subsequence: A palindrome is a nonempty string over some alphabet that reads the same forward and backward. Examples of palindromes are all strings of length 1, civic, racecar, and aibohphobia (fear of palindromes). Give an efficient algorithm to find the longest palindrome that is a subsequence of a given input string. For example, given the input character, your algorithm should return carac. What is the running time of your algorithm?
88. The Rinky Dink Company makes machines that resurface ice rinks. The demand for such products varies from month to month, and so the company needs to develop a strategy to plan its manufacturing given the fluctuating, but predictable, demand. The company wishes to design a plan for the next  $n$  months. For each month  $i$ , the company knows the demand  $d_i$ , that is, the number of machines that it will sell. Let  $D = \sum_{i=1}^n d_i$  be the total demand over the next  $n$  months. The company keeps a full-time staff who provide labor to manufacture up to  $m$  machines per month. If the company needs to make more than  $m$  machines in a given month, it can hire additional, part-time labor, at a cost that works out to  $c$  dollars per machine. Furthermore, if the company is holding any unsold machines at the end of a month, it must pay inventory costs. The company can hold up to  $D$  machines, with the cost for holding  $j$  machines given as a function  $h(j)$  for  $j = 1, 2, \dots, D$  that monotonically increases with  $j$ . Give an algorithm that calculates a plan for the company that minimizes its costs while fulfilling all the demand. The running time should be polynomial in  $n$  and  $D$ .
89. A certain string-processing programming language allows you to break a string into two pieces. Because this operation copies the string, it costs  $n$  time units to break a string of  $n$  characters into two pieces. Suppose that you want to break a string into many pieces. The order in which the breaks occur can affect the total amount of time used. For example, suppose that you want to break a 20-character string after characters 2, 8, and 10 (numbering the characters in ascending order from the left-hand end, starting from 1). If you program the breaks to occur in left-to-right order, then the first break costs 20 time units, the second break costs 18 time units (breaking the string from characters 3 to 20 at character 8), and the third break costs 12 time units, totaling 50 time units. If you program the breaks to occur in right-to-left order, however, then the first break costs 20 time units, the second break costs 10 time units, and the third break costs 8 time units, totaling 38 time units. In

yet another order, you could break first at 8 (costing 20), then break the left piece at 2 (costing another 18), and finally the right piece at 10 (costing 12), for a total cost of 40. Design an algorithm that, given the numbers of characters after which to break, determines a least-cost way to sequence those breaks. More formally, given an array  $L[1 : m]$  containing the break points for a string of  $n$  characters, compute the lowest cost for a sequence of breaks, along with a sequence of breaks that achieves this cost.